ASSIGNMENT 2 – CO2

Functions and Object-Oriented Concepts in C++

Assignment Report – Q1 to Q7 (K5 – Evaluate)

Field	Details
Name	TELAPROLU ROHITA SAI
Register Number	192425041
Course	DSA01 – C++ Programming
Submission Date	31/10/2026

Repository Structure, README and Commit Plan

Create a repository named **DSA01-<RegisterNumber>-Assignment2** and share it with the course faculty. Use one folder per task and commit after finishing each task (at least seven commits).

Folder layout

```
DSA01-<RegisterNumber>-Assignment2/
 README.md
 Q1/  q1_three_ways.cpp  q1_predict.cpp  q1_complete.cpp  screenshots/
 Q2/  q2_marks.cpp  q2_dangling.cpp  q2_vector.cpp  q2_alt.cpp  screenshots/
 Q3/  q3_ambiguous.cpp  q3_charge.cpp  q3_alt.cpp  screenshots/
 Q4/  q4_account.cpp  q4_circular.cpp  q4_deque.cpp  screenshots/
 Q5/  q5_predict.cpp  q5_complete.cpp  q5_design.cpp  q5_alt.cpp  screenshots/
 Q6/  q6_complete.cpp  q6_bench.cpp  screenshots/
 Q7/  q7_redesign.cpp  screenshots/
```

README.md (template)

```
# DSA01 Assignment 2 (CO2)
Name: <your name>        Register number: <your register number>
Compiler: g++ 13.3.0
Compile command: g++ -std=c++17 -Wall -Wextra <file>.cpp -o <file>
Benchmarks (Q6, Q7): g++ -std=c++17 -O0 and -O2

## How to run
cd Q1 && g++ -std=c++17 -Wall -Wextra q1_three_ways.cpp -o q1 && ./q1
(repeat for every file; q3_ambiguous.cpp is EXPECTED to fail to compile)

## Results
Q1: by value leaves fee 45000; reference and pointer give 40500.
Q2: returning a reference lets the caller modify the array element; reference to a local gives a warning and crashes.
Q3: ambiguity removed by using one default-argument version per weight type.
Q4: last 10 transactions kept; circular index avoids shifting.
Q5: static members give seat numbers 1-3 and a waiting list.
Q6: friend functions give 205 false; inline matters at -O2.
Q7: circular buffer removes the O(k) shifting cost.
```

Commit	Suggested message
1	Q1: call by value / reference / pointer, predict-output, complete-code
2	Q2: return by reference, dangling reference, vector invalidation
3	Q3: overload ambiguity and redesigned charge() functions
4	Q4: SavingsAccount with array inside class (shift, circular, deque)
5	Q5: static members, array of objects, Token and BusTicket
6	Q6: friend functions and inline benchmark
7	Q7: circular buffer redesign and measurements
8	README and screenshots

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

Three implementations

The concession function is written three ways. Each version is tested on its own variable that starts at 45000 with a 10% merit concession.

q1_three_ways.cpp

```
#include <iostream>
using namespace std;

// 1. Call by value: parameter is a COPY on the callee's stack frame
void concessionByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
    cout << "  inside byValue   : " << fee << '\n';
}

// 2. Call by reference: parameter is another name (alias) of the caller's variable
void concessionByRef(long long& fee, int percent)
{
    fee -= fee * percent / 100;
    cout << "  inside byRef     : " << fee << '\n';
}

// 3. Call by pointer: parameter is a copy of the ADDRESS; *fee is the caller's variable
void concessionByPtr(long long* fee, int percent)
{
    if (fee == 0) return;               // pointers can be null, so check
    *fee -= *fee * percent / 100;
    cout << "  inside byPointer : " << *fee << '\n';
}

// 4. Design decision: return the new fee (works well for chaining several concessions)
long long concessionReturn(long long fee, int percent)
{
    return fee - fee * percent / 100;
}

// const reference: read-only access to a large object without copying
struct Student {
    char name[50];
    char dept[50];
    long long fee;
    double marks[100];
};
void printFee(const Student& s)
{
    cout << "  fee of " << s.name << " = " << s.fee << '\n';
}

int main()
{
    long long f1 = 45000, f2 = 45000, f3 = 45000;

    cout << "Merit concession (10%) on 45000\n";
    concessionByValue(f1, 10);
    cout << "  stored fee after byValue   : " << f1 << '\n';

    concessionByRef(f2, 10);
    cout << "  stored fee after byRef     : " << f2 << '\n';

    concessionByPtr(&f3, 10);
    cout << "  stored fee after byPointer : " << f3 << '\n';

    cout << "\nChaining merit(10), sports(15), single parent(25) using return value\n";
    long long fee = 45000;
    fee = concessionReturn(concessionReturn(concessionReturn(fee, 10), 15), 25);
    cout << "  final fee = " << fee << '\n';

    Student s = {"Asha", "CSE", 45000, {0}};
    cout << "\nconst reference to a large object\n";
```

```
    printFee(s);
    cout << "  sizeof(Student)    = " << sizeof(Student) << " bytes (copied by value)\n";
    cout << "  sizeof(Student&)   ~ sizeof(Student*) = " << sizeof(Student*) << " bytes\n";
    cout << "  sizeof(long long)  = " << sizeof(long long) << " bytes\n";
    return 0;
}
```

Prediction before running and observed result

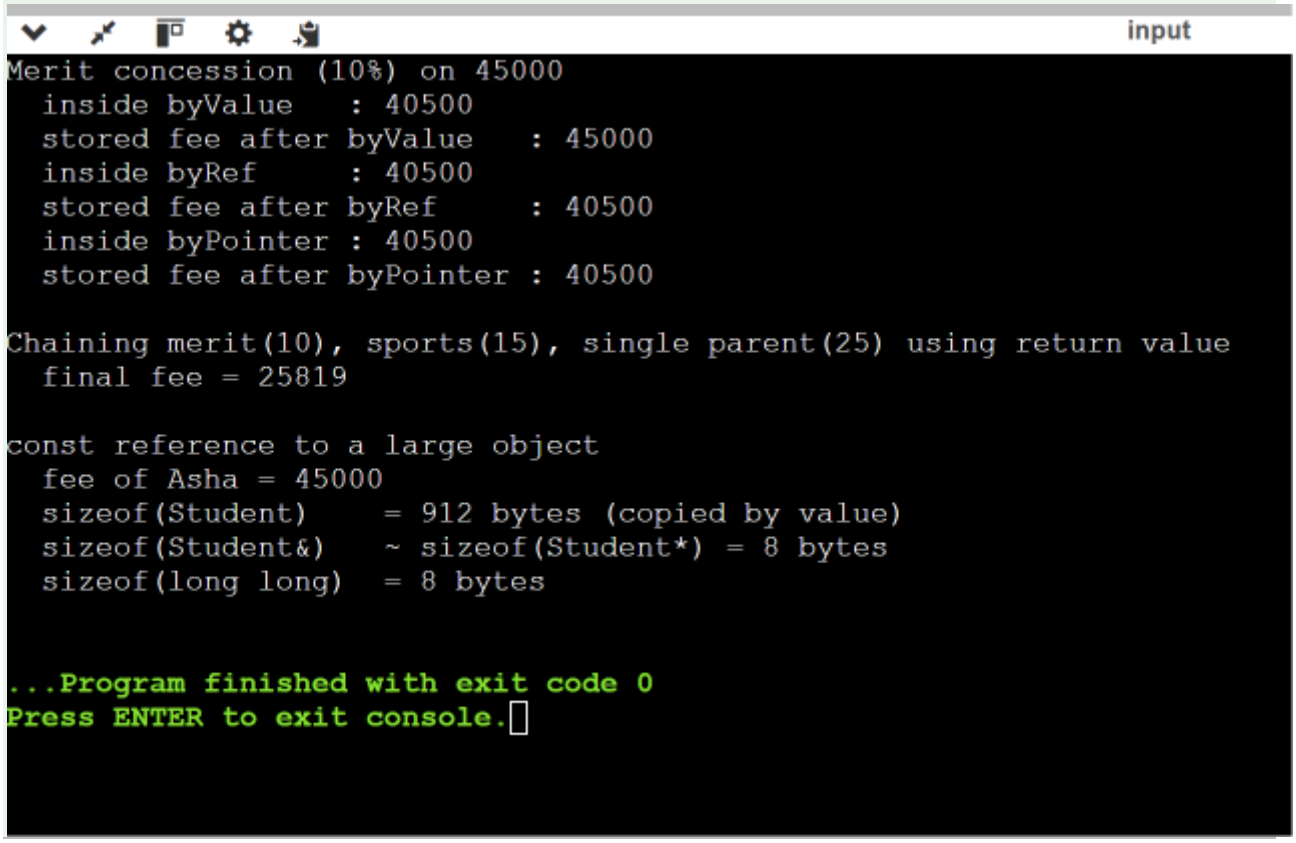
Version	Predicted stored fee	Observed	Reason
By value	45000	45000	Function changes its own copy only
By reference	40500	40500	fee is an alias of the caller's variable
By pointer	40500	40500	*fee is the caller's variable

Output of q1_three_ways.cpp

```
Merit concession (10%) on 45000
  inside byValue   : 40500
  stored fee after byValue   : 45000
  inside byRef     : 40500
  stored fee after byRef     : 40500
  inside byPointer : 40500
  stored fee after byPointer : 40500

Chaining merit(10), sports(15), single parent(25) using return value
  final fee = 25819

const reference to a large object
  fee of Asha = 45000
  sizeof(Student)    = 912 bytes (copied by value)
  sizeof(Student&)   ~ sizeof(Student*) = 8 bytes
  sizeof(long long)  = 8 bytes
```



Inquiry

- **Where each version stores its parameter.** In call by value the parameter fee is a new local variable on the function's stack frame holding a copy of 45000. In call by reference no new variable is made; fee is another name for the caller's long long (the compiler usually passes its address behind the scenes). In call by pointer the stack frame holds a new pointer variable (8 bytes) whose value is the caller's address.
- **Why only two change the caller's fee.** Only references and pointers reach the original object. The by-value copy is destroyed when the function returns, so the change disappears. The program shows the new value inside the function (40500) but the stored fee is still 45000.
- **When to use a const reference.** When the function only reads a large object (for example Student, 912 bytes in the program above). const Student& avoids the copy and the compiler rejects any attempt to modify it. For small types such as int or long long pass by value.

Design decision – change in place or return the new fee?

Decision: return the new fee (long long concessionReturn(long long fee, int percent)). Several concessions can apply one after another, and returning the value lets them be chained: merit 10%, then sports 15%, then single parent 25% gives 45000 → 40500 → 34425 → 25819 (the program printed 25819, with integer truncation at each step). Reasons:

- The function has no side effects, so it is easy to test and the office can try "what if" amounts without damaging the stored fee.
- The caller sees the assignment (fee = concessionReturn(fee, 10);), so a change to the stored fee is never hidden inside a call.
- The in-place reference version is still useful as a thin wrapper when a record should really be updated, but the calculation itself should be the pure version.

Predict the Output

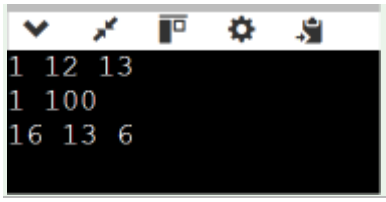
q1_predict.cpp

```
#include <iostream>
void change(int a, int& b, int* c)
{
   a += 10;
   b += 10;
   *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }
int main()
{
   int a = 1, b = 2, c = 3;
   change(a, b, &c);
   std::cout << a << ' ' << b << ' ' << c << '\n';
   bigger(a, c) = 100;
   std::cout << a << ' ' << c << '\n';
   show(1);
   show(1, 2);
   show(1, 2, 3);
   std::cout << '\n';
}
```

	Text
My predicted output (written before running)	Line 1: 1 12 13 Line 2: 1 100 Line 3: 16 13 6
Actual output	1 12 13 / 1 100 / 16 13 6 (identical, no differences)

Actual output

```
1 12 13
1 100
16 13 6
```



Which variables changed. In change(a, b, &c): a is passed by value, so the function adds 10 to a copy and a stays 1. b is a reference, so it becomes 12. c is passed through a pointer and *c += 10 makes it 13. Then bigger(a, c) compares 1 and 13 and returns a reference to the larger one, c. Because the call is on the left of =, c = 100 and a stays 1, so line 2 is 1 100.

Default arguments used by `show`. show(1) uses y = 5 and z = 10, so 1 + 5 + 10 = 16. show(1, 2) uses only z = 10, so 1 + 2 + 10 = 13. show(1, 2, 3) uses none, so 6. Each call prints a number followed by a space, which is why line 3 ends with a trailing space before the newline.

Complete the Code

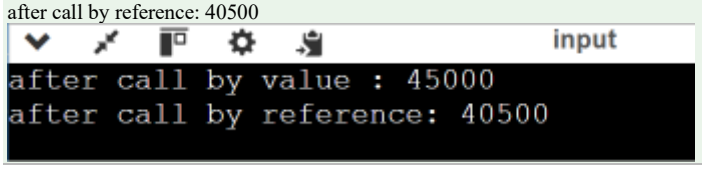
TODO 1 is completed with fee -= fee * percent / 100; (integer arithmetic) on the reference parameter.

q1_complete.cpp

```
#include <iostream>
void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100; // changes only the local copy
}
void applyConcession(long long& fee, int percent)
{
    // TODO 1 (completed): reduce the caller's fee by percent using integer arithmetic
    fee -= fee * percent / 100;
}
int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Output (matches the expected output)

```
after call by value : 45000
after call by reference: 40500
```



Why call by value leaves 45000. applyByValue receives a copy of hostelFee. It reduces that copy to 40500, then the copy is destroyed on return. The caller's variable was never touched. The reference version works on the real variable: 45000 × 10 / 100 = 4500 and 45000 − 4500 = 40500.

Alternative Concept and Complexity Analysis

Alternative: return the new fee. long long applyConcession(long long fee, int percent) { return fee - fee * percent / 100; } and the caller writes hostelFee = applyConcession(hostelFee, 10);. It needs one copy in (8 bytes) and one value out (8 bytes, usually returned in a register).

Passing style	Time per call	Space copied: long long (8 B)	Space copied: Student (912 B)
By value	O(1) for long long; O(s) for an object of size s	8 bytes	912 bytes (copy constructor runs)
By reference	O(1)	0 bytes copied (alias, address passed)	0 bytes copied
By const reference	O(1)	0 bytes copied, read-only	0 bytes copied, read-only
By pointer	O(1)	8-byte address	8-byte address
Return new value	O(1) (plus copy of the object if an object is returned)	8 bytes	912 bytes (often removed by return value optimisation)

The arithmetic is O(1) in every version. The difference is only the space copied per call, and it matters when the object is large or contains an array: copying a Student with an array of n marks costs O(n) per call, while a reference stays O(1). For a long long, by value is as cheap as a reference, so the simplest style is best.

Q2 – Return by Reference and Its Dangers (4 Marks)

Functions that return a reference to the highest mark

q2_marks.cpp

```
#include <iostream>
using namespace std;

const int N = 6;

// returns a reference to the highest element (caller can read OR modify it)
int& highestRef(int marks[], int n)
{
    int idx = 0;
    for (int i = 1; i < n; i++)
        if (marks[i] > marks[idx]) idx = i;
    return marks[idx];
}

// returns a const reference: caller can read it but not change it
const int& highestConstRef(const int marks[], int n)
{
    int idx = 0;
    for (int i = 1; i < n; i++)
        if (marks[i] > marks[idx]) idx = i;
    return marks[idx];
}

// returns a copy: caller gets a separate value
int highestCopy(const int marks[], int n)
{
    int idx = 0;
    for (int i = 1; i < n; i++)
        if (marks[i] > marks[idx]) idx = i;
    return marks[idx];
}

void printMarks(const int marks[], int n)
{
    for (int i = 0; i < n; i++) cout << marks[i] << ' ';
    cout << '\n';
}

int main()
{
    int marks[N] = {56, 78, 91, 64, 88, 72};
    cout << "Original marks      : "; printMarks(marks, N);

    highestRef(marks, N) += 2;              // moderation applied directly (call on left of =)
    cout << "After +2 moderation : "; printMarks(marks, N);

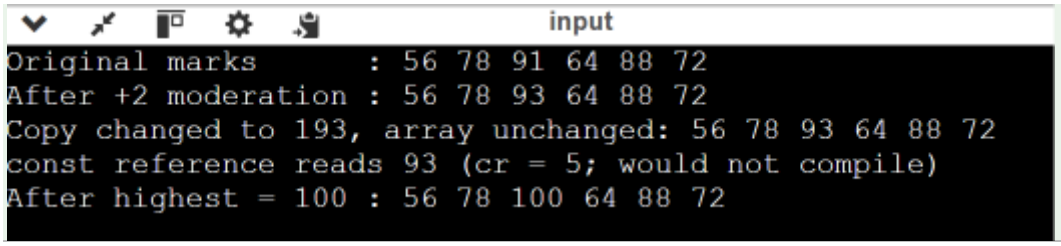
    int copy = highestCopy(marks, N);       // copy: changing it does not affect the array
    copy += 100;
    cout << "Copy changed to " << copy << ", array unchanged: "; printMarks(marks, N);

    const int& cr = highestConstRef(marks, N);
    cout << "const reference reads " << cr << " (cr = 5; would not compile)\n";

    highestRef(marks, N) = 100;             // used on the left side of =
    cout << "After highest = 100 : "; printMarks(marks, N);
    return 0;
}
```

Output

```
Original marks      : 56 78 91 64 88 72
After +2 moderation : 56 78 93 64 88 72
Copy changed to 193, array unchanged: 56 78 93 64 88 72
const reference reads 93 (cr = 5; would not compile)
After highest = 100 : 56 78 100 64 88 72
```



highestRef(marks, N) += 2; changes the array element directly because the function returns the element itself (an lvalue). The same call can appear on the left of = (highestRef(marks, N) = 100;).

Return type	What the caller gets	Can it change the array?
int&	An alias of the array element	Yes – f() += 2; or f() = 100;
const int&	A read-only alias of the element	No – assigning is a compile error; no copy is made
int	A copy of the value	No – changing the copy leaves the array unchanged (193 vs 93 above)

Returning a reference to a local variable

q2_dangling.cpp

```
#include <iostream>
using namespace std;

int& badReference()
{
    int local = 99;
    return local;          // WARNING: reference to local variable returned
}

int main()
{
    int& r = badReference();   // r refers to a variable that no longer exists
    cout << "Value through dangling reference: " << r << '\n';   // undefined behaviour
    return 0;
}
```

Compiler warning (g++ -std=c++17 -Wall -Wextra)

```
q2_dangling.cpp: In function 'int& badReference()':
q2_dangling.cpp:7:12: warning: reference to local variable 'local' returned [-Wreturn-local-addr]
    7 |     return local;          // WARNING: reference to local variable returned
      |            ^~~~~
q2_dangling.cpp:6:9: note: declared here
    6 |     int local = 99;
```



Run result

```
exit code: 139
```

Why it is undefined behaviour. local lives on the stack frame of badReference. When the function returns, the frame is released and the variable no longer exists, yet the reference still names that location. Reading it (a dangling reference) may give garbage, the old value or a crash. With g++ 13 at -O0 the program crashed with exit code 139 (segmentation fault), because the compiler replaced the returned address by a null address after warning about it. The C++ rules do not guarantee any particular behaviour, so such code must never be used.

Changed condition – a std::vector that can grow

q2_vector.cpp

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
   vector<int> marks;
   marks.push_back(56);
   marks.push_back(78);
   marks.push_back(91);

   int* before = &marks[0];              // address of first element now
   cout << "capacity before growth : " << marks.capacity() << '\n';

   // r = reference obtained BEFORE the vector grows
   int& r = marks[0];
   (void)r;

   size_t oldCap = marks.capacity();
   while (marks.capacity() == oldCap)
      marks.push_back(60);               // keep adding until re-allocation happens

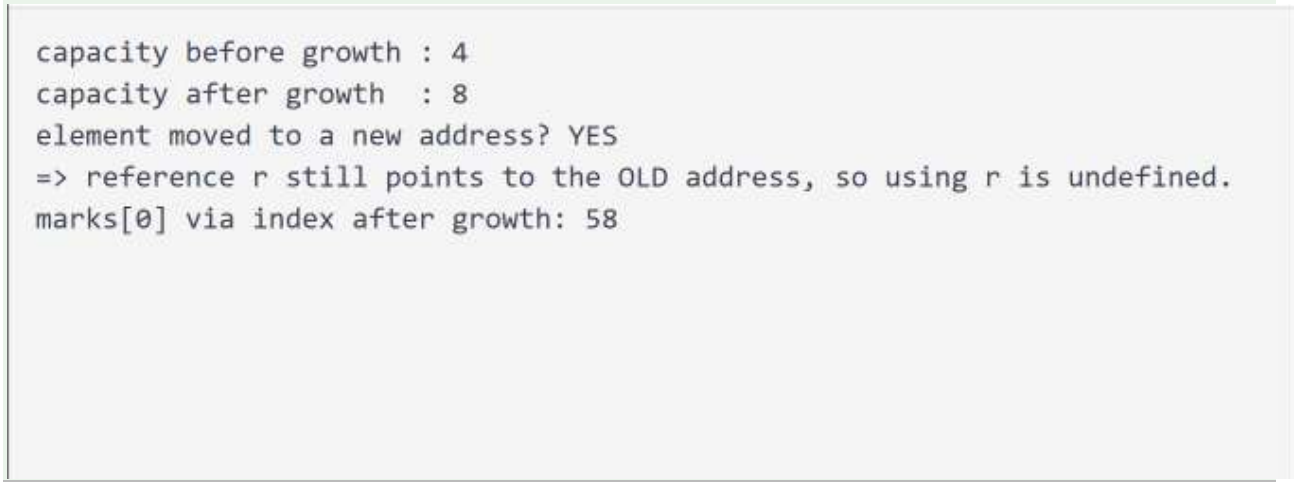
   cout << "capacity after growth  : " << marks.capacity() << '\n';
   cout << "element moved to a new address? "
        << (before != &marks[0] ? "YES" : "NO") << '\n';
   cout << "=> reference r still points to the OLD address, so using r is undefined.\n";

   // safe approach: remember the INDEX, not the reference
   int idx = 0;
```

```
    marks[idx] += 2;
    cout << "marks[0] via index after growth: " << marks[idx] << '\n';
    return 0;
}
```

Output

```
capacity before growth : 4
capacity after growth  : 8
element moved to a new address? YES
=> reference r still points to the OLD address, so using r is undefined.
marks[0] via index after growth: 58
```



A vector keeps its elements in one block of memory. When push_back needs more room than the capacity, the vector allocates a bigger block, copies the elements into it and frees the old block. In the output the capacity grew from 4 to 8 and the first element moved to a new address. Any reference, pointer or iterator obtained before the growth still points to the freed block, so using it is undefined behaviour. The safe approach is to hold an index (or call the function again) instead of keeping a reference across operations that may add elements.

Alternative Concept and Complexity Analysis

If the function returns an index (int) or a pointer (int*) instead of a reference, the calling code changes as follows.

q2_alt.cpp

```
#include <iostream>
#include <vector>
using namespace std;

int highestIndex(const vector<int>& v)
{
    int idx = 0;
    for (size_t i = 1; i < v.size(); i++)
        if (v[i] > v[idx]) idx = (int)i;
    return idx;
}

int* highestPtr(vector<int>& v)
{
    int* p = &v[0];
    for (size_t i = 1; i < v.size(); i++)
        if (v[i] > *p) p = &v[i];
    return p;
}

int main()
{
    vector<int> v = {56, 78, 91, 64};

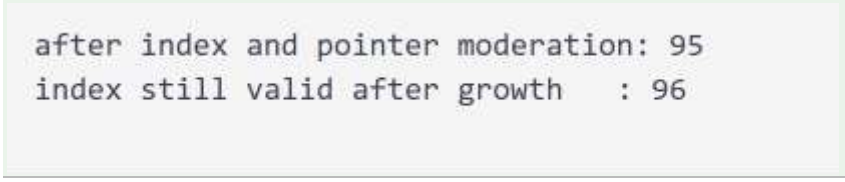
    int idx = highestIndex(v);          // index version
    v[idx] += 2;
```

```
    int* p = highestPtr(v);              // pointer version
    *p += 2;
    cout << "after index and pointer moderation: " << v[idx] << '\n';

    v.push_back(10);v.push_back(11);v.push_back(12);v.push_back(13);  // vector may grow
    v[idx] += 1;                         // index is STILL valid after growth
    cout << "index still valid after growth   : " << v[idx] << '\n';
    // *p += 1;   // would be UNSAFE: p may point to freed memory
    return 0;
}
```

Output

```
after index and pointer moderation: 95
index still valid after growth   : 96
```



Version	Calling code	Time	Space	Safety when array becomes a growing vector
Reference int&	highest(v) += 2;	O(n)	O(1)	Safe if used immediately; dangling if stored across a push_back
Index int	v[highestIndex(v)] += 2;	O(n)	O(1)	Safe – an index stays valid after growth (as long as the element is not removed)
Pointer int*	*highestPtr(v) += 2;	O(n)	O(1)	Unsafe after growth, can also be null; must be checked

All three need one pass over n marks, so the time is O(n) and the extra space is O(1). The reference gives the neatest syntax, but the index is the most robust choice when the container may grow.

Q3 – Default Arguments and Function Overloading (6 Marks)

The compiler error (ambiguous call)

q3_ambiguous.cpp (original design)

```
// ORIGINAL (faulty) design from the problem statement - does NOT compile
#include <iostream>
using namespace std;

double charge(double weight)                              { return weight * 50; }
double charge(double weight, bool express)                { return weight * 50 + (express ? 30 : 0); }
double charge(double weight, bool express = false, bool insured = false)
{
   return weight * 50 + (express ? 30 : 0) + (insured ? 20 : 0);
}

int main()
{
   cout << charge(2.0) << '\n';          // matches version 1 AND version 3 -> ambiguous
   cout << charge(2.0, true) << '\n';    // matches version 2 AND version 3 -> ambiguous
}
```

Compiler error

```
q3_ambiguous.cpp: In function 'int main()':
q3_ambiguous.cpp:14:19: error: call of overloaded 'charge(double)' is ambiguous
   14 |     cout << charge(2.0) << '\n';          // matches version 1 AND version 3 -> ambiguous
      |             ~~~~~~^~~~~
q3_ambiguous.cpp:5:8: note: candidate: 'double charge(double)'
    5 | double charge(double weight)                              { return weight * 50; }
      |        ^~~~~~
q3_ambiguous.cpp:7:8: note: candidate: 'double charge(double, bool, bool)'
    7 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
q3_ambiguous.cpp:15:19: error: call of overloaded 'charge(double, bool)' is ambiguous
   15 |     cout << charge(2.0, true) << '\n';    // matches version 2 AND version 3 -> ambiguous
      |             ~~~~~~^~~~~~~~~~
q3_ambiguous.cpp:6:8: note: candidate: 'double charge(double, bool)'
    6 | double charge(double weight, bool express)                { return weight * 50 + (express ? 30 : 0); }
      |        ^~~~~~
q3_ambiguous.cpp:7:8: note: candidate: 'double charge(double, bool, bool)'
    7 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
```

Explanation. For charge(2.0) two functions are equally good: charge(double) is an exact match, and charge(double, bool = false, bool = false) is also an exact match because the missing arguments are filled by defaults. The compiler has no rule to prefer one, so the call is ambiguous. The same happens for charge(2.0, true) between the two-parameter version and the default-argument version. The first two functions were already covered by the third, so they should be removed.

Redesigned charge functions

Design: one function per kind of input, each with default arguments for the optional flags, so that common calls are short and no two functions accept the same call. Grams (int) and kilograms (double) are different types and are therefore different overloads; the Parcel overload takes one object.

q3_charge.cpp

```
#include <iostream>
using namespace std;

const double RATE_PER_KG   = 50.0;   // Rs per kg
const double EXPRESS_FEE   = 30.0;
const double INSURANCE_FEE = 20.0;

struct Parcel {
   double weightKg;
   bool   express;
   bool   insured;
};
```

```
// (1) weight in kg as double: defaults only on the right
double charge(double weightKg, bool express = false, bool insured = false)
{
   double total = weightKg * RATE_PER_KG;
   if (express) total += EXPRESS_FEE;
   if (insured) total += INSURANCE_FEE;
   return total;
}

// (2) weight in grams as int: converts and reuses version (1)
double charge(int grams, bool express = false, bool insured = false)
{
   return charge(grams / 1000.0, express, insured);
}

// (3) a Parcel object
double charge(const Parcel& p)
{
   return charge(p.weightKg, p.express, p.insured);
}

int main()
{
   Parcel p = {3.0, true, true};

   cout << "call 1  charge(2)           = " << charge(2)           << "   (int -> grams version)\n";
   cout << "call 2  charge(2.0f)        = " << charge(2.0f)        << "   (float -> double by promotion -> kg version)\n";
   cout << "call 3  charge(2.5)         = " << charge(2.5)         << "   (double exact -> kg version)\n";
   cout << "call 4  charge(1500, true)  = " << charge(1500, true)  << "   (int exact -> grams version, express)\n";
   cout << "call 5  charge(1.5,true,true)= " << charge(1.5, true, true) << "   (double -> kg, express+insured)\n";
   cout << "call 6  charge(p)           = " << charge(p)           << "   (Parcel version)\n";
   return 0;
}
```

Predictions and observed selections

#	Call	Predicted function	Observed value	Reason
1	charge(2)	int (grams)	0.1	int is an exact match for the grams version; 2 g = 0.002 kg
2	charge(2.0f)	double (kg)	100	float→double is a promotion; float→int is only a conversion, so promotion wins
3	charge(2.5)	double (kg)	125	exact match
4	charge(1500, true)	int (grams), express	105	exact match on int; 1.5 kg × 50 + 30
5	charge(1.5, true, true)	double (kg), both options	125	75 + 30 + 20
6	charge(p)	Parcel	200	3 kg × 50 + 30 + 20

Output of q3_charge.cpp

```
call 1  charge(2)           = 0.1   (int -> grams version)
call 2  charge(2.0f)        = 100   (float -> double by promotion -> kg version)
call 3  charge(2.5)         = 125   (double exact -> kg version)
call 4  charge(1500, true)  = 105   (int exact -> grams version, express)
call 5  charge(1.5,true,true)= 125   (double -> kg, express+insured)
call 6  charge(p)           = 200   (Parcel version)
```

```
call 1  charge(2)             = 0.1   (int -> grams version)
call 2  charge(2.0f)          = 100   (float -> double by promotion -> kg
version)
call 3  charge(2.5)           = 125   (double exact -> kg version)
call 4  charge(1500, true)    = 105   (int exact -> grams version, express)
call 5  charge(1.5,true,true)= 125   (double -> kg, express+insured)
call 6  charge(p)             = 200   (Parcel version)
```

All six predictions matched the observed selections. Note that charge(2) means 2 **grams**; to charge 2 kg the caller writes charge(2.0). This is a design trade-off of overloading on int and double, so the documentation must state the unit of each overload.

Inquiry

- **How the compiler chooses.** It first collects the functions with the right name and a matching number of arguments (default arguments allow fewer). For each argument it ranks the conversion: (1) exact match (including adding const or reference binding), (2) promotion (float→double, char/short→int, bool→int), (3) standard conversion (int→double, double→int, int→bool), (4) user-defined conversion. The function that is better in at least one argument and not worse in any other is chosen. If there is no single best function the call is ambiguous.
- **Why defaults must be given from the right.** Arguments are matched left to right, so a call can only leave out the trailing ones. If f(int a = 1, int b) were allowed, the call f(5) could not say which parameter 5 belongs to and b would have no value. Therefore once one parameter has a default, all parameters to its right must have defaults.

Alternative Concept and Complexity Analysis

Alternative: a single Parcel structure with optional fields (and a function template for other weight types).

q3_alt.cpp

```
// Alternative design: ONE function taking a Parcel with optional fields
#include <iostream>
using namespace std;

struct Parcel {
    double weightKg;
    bool   express   = false;     // optional fields have default values
    bool   insured   = false;
    // adding a new option later (e.g. bool fragile = false;) needs no change at the call sites
};

double charge(const Parcel& p)
{
    double total = p.weightKg * 50.0;
    if (p.express) total += 30.0;
    if (p.insured) total += 20.0;
    return total;
}

// a function template for different weight types
template <typename W>
double chargeWeight(W w) { return charge(Parcel{static_cast<double>(w)}); }

int main()
{
    cout << charge(Parcel{2.5}) << '\n';              // 125
```

```
    cout << charge(Parcel{2.5, true}) << '\n';          // 155
    cout << charge(Parcel{2.5, true, true}) << '\n';    // 175
    cout << chargeWeight(2) << ' ' << chargeWeight(2.5f) << '\n';   // template: int and float both work
}
```

Output

```
125
155
175
100 125
```

```
125
155
175
100 125
```

Aspect	Overloads with default arguments	Single Parcel / template
Overload resolution time	Compile time only, no run-time cost	Compile time only (templates are instantiated at compile time)
Run time	O(1)	O(1)
Space copied per call	Arguments by value: 8 + 1 + 1 bytes (plus padding)	Parcel passed by const reference: 0 bytes copied (the object is 16 bytes)
Adding a new option (e.g. fragile)	Add another parameter to every overload; risk of new ambiguity and all call sites that use positional flags are fragile (charge(2.0, true, false, true))	Add one field with a default value; existing calls keep working
Readability of calls	Short for common calls, but flags are positional booleans	Longer call (Parcel{2.5, true}) but each field has a name

Q4 – Specifying a Class with an Array Inside It (6 Marks)

Class specification (shifting array)

SavingsAccount keeps the account number, balance, an array of 10 Transaction records and a count as **private** data. The constructor and getBalance() are defined inside the class; deposit(), withdraw() and miniStatement() are defined outside using the scope resolution operator SavingsAccount::.

q4_account.cpp

```
#include <iostream>
using namespace std;

const int MAXTX = 10;          // number of transactions remembered

struct Transaction {
    char type;                 // 'D' deposit, 'W' withdrawal
    int  amount;
    int  balanceAfter;
};

class SavingsAccount {
private:
    int         accNo;
    int         balance;
    Transaction last[MAXTX];   // private array: only the class keeps it consistent
    int         count;         // how many entries are valid (0..MAXTX)

    // shift version: when full, discard the oldest by moving every element left (O(k))
    void record(char type, int amount)
    {
        if (count == MAXTX) {
            for (int i = 1; i < MAXTX; i++)
                last[i - 1] = last[i];
            count--;
        }
        last[count].type = type;
        last[count].amount = amount;
        last[count].balanceAfter = balance;
        count++;
    }

public:
    // constructor and getter defined INSIDE the class
    SavingsAccount(int no, int opening) : accNo(no), balance(opening), count(0) {}
    int getBalance() const { return balance; }

    // these three are defined OUTSIDE using the scope resolution operator
    void deposit(int amount);
    bool withdraw(int amount);
    void miniStatement() const;
};

void SavingsAccount::deposit(int amount)
{
    if (amount <= 0) return;
    balance += amount;
    record('D', amount);
}

bool SavingsAccount::withdraw(int amount)
{
    if (amount <= 0 || amount > balance) {
        cout << "  withdraw " << amount << " refused (insufficient balance)\n";
        return false;
    }
    balance -= amount;
    record('W', amount);
    return true;
}

void SavingsAccount::miniStatement() const
{
    cout << "Mini statement for account " << accNo << " (last " << count << " transactions)\n";
```

```
    for (int i = count - 1; i >= 0; i--)      // newest first
        cout << "  " << (last[i].type == 'D' ? "Deposit   " : "Withdrawal")
             << " " << last[i].amount
             << "   balance " << last[i].balanceAfter << '\n';
    cout << "Current balance: " << balance << '\n';
}

int main()
{
    SavingsAccount acc(1001, 5000);
    for (int i = 1; i <= 8; i++) acc.deposit(i * 100);     // 8 deposits
    acc.withdraw(250);
    acc.withdraw(1000000);                          // refused
    acc.withdraw(400);
    acc.deposit(5000);
    acc.withdraw(750);                              // 12 successful transactions in total
    acc.miniStatement();                            // only the last 10 are shown
    return 0;
}
```

Output (12 successful transactions made, the last 10 shown)

```
  withdraw 1000000 refused (insufficient balance)
Mini statement for account 1001 (last 10 transactions)
  Withdrawal 750   balance 12200
  Deposit    5000   balance 12950
  Withdrawal 400   balance 7950
  Withdrawal 250   balance 8350
  Deposit    800   balance 8600
  Deposit    700   balance 7800
  Deposit    600   balance 7100
  Deposit    500   balance 6500
  Deposit    400   balance 6000
  Deposit    300   balance 5600
Current balance: 12200
```


```
  withdraw 1000000 refused (insufficient balance)
Mini statement for account 1001 (last 10 transactions)
  Withdrawal 750   balance 12200
  Deposit    5000   balance 12950
  Withdrawal 400   balance 7950
  Withdrawal 250   balance 8350
  Deposit    800   balance 8600
  Deposit    700   balance 7800
  Deposit    600   balance 7100
  Deposit    500   balance 6500
  Deposit    400   balance 6000
  Deposit    300   balance 5600
Current balance: 12200
```


Twelve successful transactions were made (8 deposits, 2 withdrawals, 1 deposit, 1 withdrawal). The refused withdrawal of 1000000 is not recorded. The mini statement shows only the last 10, newest first, so the two oldest deposits (100 and 200) were discarded.

Inquiry

- **Why the array must be private.** The class must keep the rules (count between 0 and 10, entries in time order, balanceAfter consistent with the balance). If the array were public any code could overwrite or delete a transaction without changing the balance, which breaks the statement and the audit trail. Private data also lets the class change its internal storage (shift to circular) without changing any calling code.

- **The 11th transaction with shifting.** When count == 10 the function moves entries 1..9 to positions 0..8 (9 copies) and then stores the new one at position 9. Cost: O(k) element moves per transaction for k = 10 (O(1) space extra).
- **The 11th transaction with a circular index.** A start index marks the oldest entry. The new transaction overwrites the oldest slot and start moves forward by one (modulo 10). Cost: O(1) per transaction.

q4_circular.cpp (circular buffer version)

```
#include <iostream>
using namespace std;

const int MAXTX = 10;

struct Transaction { char type; int amount; int balanceAfter; };

class SavingsAccount {
private:
    int         accNo, balance;
    Transaction last[MAXTX];
    int         start;       // index of the OLDEST entry
    int         count;

    // circular version: O(1) - overwrite the oldest slot and advance start
    void record(char type, int amount)
    {
        int pos = (start + count) % MAXTX;
        if (count == MAXTX) {
            pos = start;
            start = (start + 1) % MAXTX;
        } else {
            count++;
        }
        last[pos].type = type;
        last[pos].amount = amount;
        last[pos].balanceAfter = balance;
    }

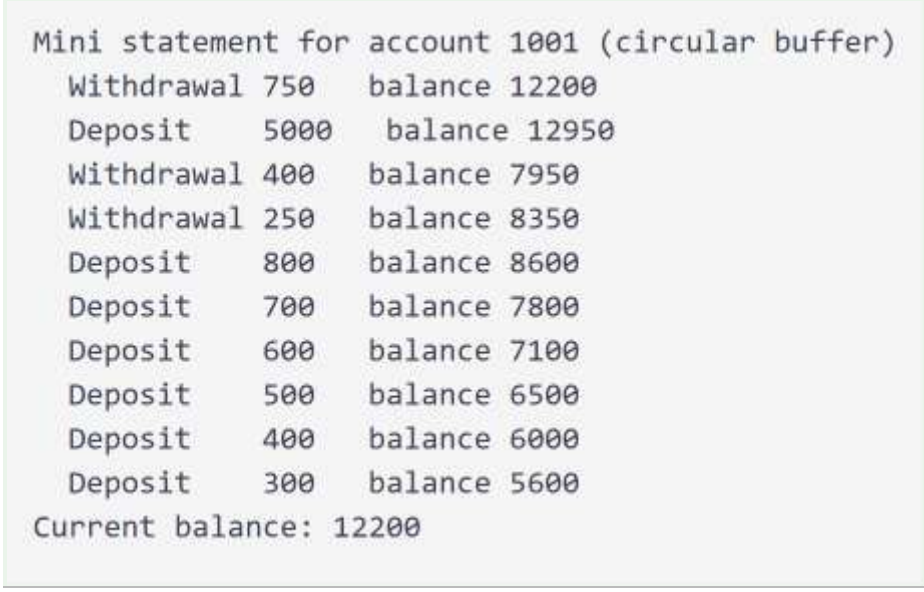
public:
    SavingsAccount(int no, int opening) : accNo(no), balance(opening), start(0), count(0) {}
    void deposit(int amount);
    bool withdraw(int amount);
    void miniStatement() const;
};

void SavingsAccount::deposit(int amount) { if (amount > 0) { balance += amount; record('D', amount); } }
bool SavingsAccount::withdraw(int amount)
{
    if (amount <= 0 || amount > balance) return false;
    balance -= amount; record('W', amount); return true;
}
void SavingsAccount::miniStatement() const
{
    cout << "Mini statement for account " << accNo << " (circular buffer)\n";
    for (int i = count - 1; i >= 0; i--) {
        const Transaction& t = last[(start + i) % MAXTX];
        cout << "  " << (t.type == 'D' ? "Deposit   " : "Withdrawal")
             << " " << t.amount << "   balance " << t.balanceAfter << '\n';
    }
    cout << "Current balance: " << balance << '\n';
}

int main()
{
    SavingsAccount acc(1001, 5000);
    for (int i = 1; i <= 8; i++) acc.deposit(i * 100);
    acc.withdraw(250);
    acc.withdraw(1000000);
    acc.withdraw(400);
    acc.deposit(5000);
    acc.withdraw(750);
    acc.miniStatement();
    return 0;
}
```

Output (identical statement)

```
Mini statement for account 1001 (circular buffer)
  Withdrawal 750   balance 12200
  Deposit    5000   balance 12950
  Withdrawal 400   balance 7950
  Withdrawal 250   balance 8350
  Deposit    800   balance 8600
  Deposit    700   balance 7800
  Deposit    600   balance 7100
  Deposit    500   balance 6500
  Deposit    400   balance 6000
  Deposit    300   balance 5600
Current balance: 12200
```



Design decision – last 50 transactions

The circular buffer changes least. In both designs the constant MAXTX changes from 10 to 50 and nothing else in the class changes. But the cost is different: with shifting each new transaction now copies 49 entries instead of 9 (about 5.4 times more work), while the circular buffer still does one write. The public functions (deposit, withdraw, miniStatement) are unchanged, so no caller changes, and no extra memory is needed beyond one start index. The array stays private and the order of the printed statement is unchanged.

Alternative Concept and Complexity Analysis

Alternative: `std::deque`. push_back adds the newest transaction and pop_front removes the oldest when the size exceeds 10.

q4_deque.cpp

```
#include <iostream>
#include <deque>
using namespace std;

struct Transaction { char type; int amount; int balanceAfter; };

class SavingsAccount {
   int accNo, balance;
   deque<Transaction> last;          // push_back newest, pop_front oldest
   static const size_t MAXTX = 10;
public:
   SavingsAccount(int no, int opening) : accNo(no), balance(opening) {}
   void deposit(int amount)
   {
      balance += amount;
      last.push_back({'D', amount, balance});
      if (last.size() > MAXTX) last.pop_front();     // O(1)
   }
   void withdraw(int amount)
   {
```

```
        if (amount > balance) return;
        balance -= amount;
        last.push_back({'W', amount, balance});
        if (last.size() > MAXTX) last.pop_front();
    }
    void miniStatement() const
    {
        cout << "Mini statement (deque) - " << last.size() << " entries\n";
        for (auto it = last.rbegin(); it != last.rend(); ++it)
            cout << "  " << it->type << " " << it->amount << "  balance " << it->balanceAfter << '\n';
    }
};

int main()
{
    SavingsAccount acc(1001, 5000);
    for (int i = 1; i <= 12; i++) acc.deposit(i * 100);
    acc.withdraw(250);
    acc.miniStatement();
}
```

Output

```
Mini statement (deque) - 10 entries
  W 250  balance 12550
  D 1200  balance 12800
  D 1100  balance 11600
  D 1000  balance 10500
  D 900  balance 9500
  D 800  balance 8600
  D 700  balance 7800
  D 600  balance 7100
  D 500  balance 6500
  D 400  balance 6000
```


```
Mini statement (deque) - 10 entries
  W 250  balance 12550
  D 1200  balance 12800
  D 1100  balance 11600
  D 1000  balance 10500
  D 900  balance 9500
  D 800  balance 8600
  D 700  balance 7800
  D 600  balance 7100
  D 500  balance 6500
  D 400  balance 6000
```


Design	Time to add a transaction	Space	Comments
Fixed array with shifting	O(k) once full (k − 1 moves)	O(k) fixed, inside the object	Simplest code; slows down as k grows
Circular buffer (start index)	O(1)	O(k) fixed, inside the object, plus one int	Needs modulo arithmetic; fixed maximum size
std::deque	O(1) (amortised)	O(k) plus block bookkeeping, heap allocation	Flexible size; elements not contiguous; heap allocations

Measured evidence for the shifting and circular designs is given in Q7 (for example, 1,000,000 deposits at k = 5000 take about 15.7 s with shifting and 6.3 ms with the circular index on this machine).

Q5 – Static Members and Arrays of Objects (7 Marks)

From the principle to a class design

Principle: data that belongs to the whole class exists only once. Applying it to the bus tickets:

- **Static data members:** issued (seats given so far), waiting (number on the waiting list) and CAPACITY (seats in the bus). Each is the same for every ticket, so one copy is shared. seat, waitNo and name stay non-static because each ticket has its own.
- **Where a static data member is defined:** the class only declares it. It must be defined once outside the class, in one source file, for example int BusTicket::issued = 0;. If it were defined in a header included in many files the linker would report multiple definitions. (static const int CAPACITY = 4; with an integer constant can be given its value inside the class.)
- **Why a static member function cannot use `this` or non-static members:** a static function belongs to the class and can be called without any object (BusTicket::seatsIssued()). Without an object there is no this, so there is no seat or name to refer to. It can use only static members.
- **How an array of objects is created:** BusTicket trip[6] = {BusTicket("Asha"), ...}; runs the constructor once for every element in order of index. Without an initialiser list a default constructor is needed (here name has a default argument).
- **Count in a loop or a function:** the static counter belongs to the class, not to the object, so it is not reset when a temporary object is destroyed. Creating a ticket inside a function or loop increases the count every time, and the count is not decreased unless a destructor is written to do it.

q5_design.cpp (test design: array larger than the capacity)

```
#include <iostream>
#include <string>
using namespace std;

class BusTicket {
private:
   static int issued;             // seats issued so far (one copy for the whole class)
   static int waiting;            // students on the waiting list
   static const int CAPACITY = 4;  // fixed seats in the bus
   int seat;                      // 0 means waiting list
   int waitNo;
   string name;
public:
   BusTicket(const string& n = "unknown") : seat(0), waitNo(0), name(n)
   {
      if (issued < CAPACITY) { seat = ++issued; }
      else                   { waitNo = ++waiting; }
   }
   static int seatsIssued()  { return issued; }     // static function: no 'this'
   static int waitingCount() { return waiting; }
   static int seatsLeft()    { return CAPACITY - issued; }
   void print() const
   {
      cout << "  " << name << ": ";
      if (seat) cout << "Seat " << seat << '\n';
      else      cout << "Waiting list #" << waitNo << '\n';
   }
};
int BusTicket::issued  = 0;        // static data members are DEFINED once outside the class
int BusTicket::waiting = 0;

void bookInsideFunction()
{
   BusTicket temp("TempStudent");  // destroyed at the end of the function
   temp.print();
}

int main()
{
   BusTicket trip[6] = {BusTicket("Asha"), BusTicket("Bala"), BusTicket("Chitra"),
                        BusTicket("Dev"),  BusTicket("Esha"), BusTicket("Farid")};
   cout << "Array of 6 tickets, capacity 4:\n";
   for (int i = 0; i < 6; i++) trip[i].print();
   cout << "Seats issued: " << BusTicket::seatsIssued()
        << ", waiting: " << BusTicket::waitingCount()
```

```
        << ", seats left: " << BusTicket::seatsLeft() << '\n';

    cout << "\nTickets created inside a function called twice:\n";
    bookInsideFunction();
    bookInsideFunction();
    cout << "Seats issued: " << BusTicket::seatsIssued()
         << ", waiting: " << BusTicket::waitingCount()
         << "  (objects were destroyed but the static counters were not reduced)\n";

    cout << "\nTickets created inside a loop:\n";
    for (int i = 0; i < 2; i++) { BusTicket t("LoopStudent"); }
    cout << "Waiting count is now: " << BusTicket::waitingCount() << '\n';
    return 0;
}
```

Output

```
Array of 6 tickets, capacity 4:
  Asha: Seat 1
  Bala: Seat 2
  Chitra: Seat 3
  Dev: Seat 4
  Esha: Waiting list #1
  Farid: Waiting list #2
Seats issued: 4, waiting: 2, seats left: 0

Tickets created inside a function called twice:
  TempStudent: Waiting list #3
  TempStudent: Waiting list #4
Seats issued: 4, waiting: 4  (objects were destroyed but the static counters were not reduced)

Tickets created inside a loop:
Waiting count is now: 6
```


```
Array of 6 tickets, capacity 4:
  Asha: Seat 1
  Bala: Seat 2
  Chitra: Seat 3
  Dev: Seat 4
  Esha: Waiting list #1
  Farid: Waiting list #2
Seats issued: 4, waiting: 2, seats left: 0

Tickets created inside a function called twice:
  TempStudent: Waiting list #3
  TempStudent: Waiting list #4
Seats issued: 4, waiting: 4  (objects were destroyed but the static counters
were not reduced)

Tickets created inside a loop:
Waiting count is now: 6
```


Verification. Capacity is 4 and six tickets were created: seats 1–4 were issued and Esha and Farid went to waiting-list positions 1 and 2. The function was called twice and each call created one ticket; both went to the waiting list (numbers 3 and 4) and the counters did not go back when the objects were destroyed. The loop that created two more temporary tickets raised the waiting count to 6.

Predict the Output

q5_predict.cpp

```
#include <iostream>
class Token {
    static int next;
    int id;
public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};
int Token::next = 100;
int main()
{
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get() << ' ' << Token::issued() << '\n';
}
```

	Text
My predicted output (written before running)	101 103 104 4
Actual output	101 103 104 4 (no differences)

Actual output

```
101 103 104 4
```

```
101 103 104 4
```

Why the counter starts at 100 and how many times the constructor ran. The definition int Token::next = 100; creates the single static variable and gives it the value 100 before main starts. Each Token constructor does ++next. The constructor ran 4 times: three times for counter[3] (ids 101, 102, 103) and once for extra (id 104). So counter[0].get() is 101, counter[2].get() is 103, extra.get() is 104 and Token::issued() is 104 − 100 = 4. issued() is a static function, so it was called with the class name and no object.

Complete the Code

TODO 1 increases issued and gives the ticket the next seat when fewer than CAPACITY seats are issued. TODO 2 returns issued.

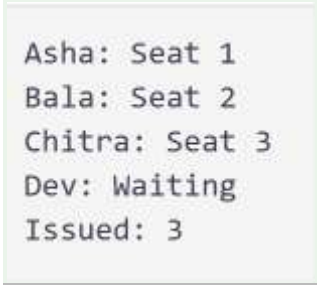
q5_complete.cpp

```
#include <iostream>
#include <string>
class BusTicket {
    static int issued;              // shared by all BusTicket objects
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        // TODO 1 (completed)
        if (issued < CAPACITY) {
            issued++;
            seat = issued;
        }
    }
    static int count()
    {
        // TODO 2 (completed)
        return issued;
    }
    void print() const
    {
```

```
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting")) << '\n';
    }
};
int BusTicket::issued = 0;          // definition of the static data member
int main()
{
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"), BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Output (matches the expected output)

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```



Tickets created inside a function called twice. The static counter lives for the whole program, so each call continues from the previous count. With capacity 3, the first call would take the next free seat and the second call the following one (or the waiting list when full); destroying the local ticket does not give the seat back. This was demonstrated in q5_design.cpp above, where the counters stayed at their increased values after the function returned.

Alternative Concept and Complexity Analysis

Alternative: a separate `TicketOffice` object passed to every `BusTicket` (a global variable would work the same way but with no protection).

q5_alt.cpp

```
// Alternative: a separate TicketOffice object passed to every BusTicket
#include <iostream>
#include <string>
using namespace std;

class TicketOffice {
public:
    int issued;
    int capacity;
    TicketOffice(int cap) : issued(0), capacity(cap) {}
};

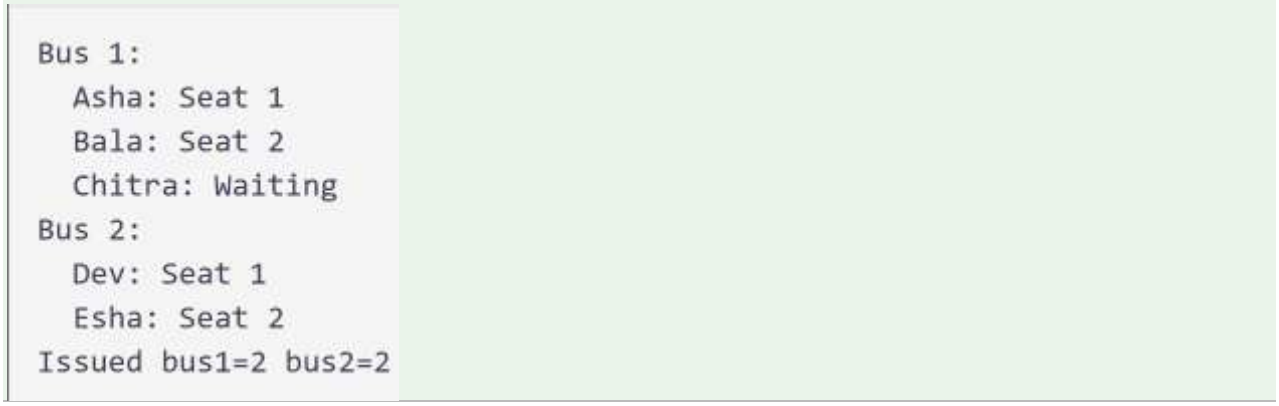
class BusTicket {
    int seat;
    string name;
public:
    BusTicket(const string& n, TicketOffice& office) : seat(0), name(n)
    {
        if (office.issued < office.capacity) seat = ++office.issued;
    }
    void print() const { cout << "  " << name << ": " << (seat ? "Seat " + to_string(seat) : string("Waiting")) << '\n'; }
};

int main()
{
    TicketOffice bus1(2), bus2(3);                  // two buses at the same time
    BusTicket a("Asha", bus1), b("Bala", bus1), c("Chitra", bus1);
    BusTicket d("Dev", bus2),  e("Esha", bus2);
    cout << "Bus 1:\n"; a.print(); b.print(); c.print();
```

```
    cout << "Bus 2:\n"; d.print(); e.print();
    cout << "Issued bus1=" << bus1.issued << " bus2=" << bus2.issued << '\n';
}
```

Output (two buses at the same time)

```
Bus 1:
  Asha: Seat 1
  Bala: Seat 2
  Chitra: Waiting
Bus 2:
  Dev: Seat 1
  Esha: Seat 2
Issued bus1=2 bus2=2
```



Design	Time per ticket	Space	Encapsulation	Two buses at once
Static data member	O(1)	One counter for the whole class	Good – counter is private to the class	No – one counter for all tickets
Global variable	O(1)	One counter	Poor – any code can change it	No
TicketOffice object	O(1)	One counter per office (plus a reference per call)	Good if the counter is private	Yes – one office per bus

The time is O(1) in all designs. The static member is the simplest when there is exactly one bus. If the college runs several buses, the TicketOffice design is better because each bus keeps its own counter.

Q6 – Friend Functions and Inline Functions (8 Marks)

Design of the comparison

- **Why a friend function.** minutesBetween(a, b) and earlier(a, b) treat both times equally. A member function would be called as a.minutesTo(b), which makes one object special and gives it access to the other one only through its own member code. A friend function is an ordinary function that is given permission to read the private h and m of both objects, so the call is symmetric: minutesBetween(a, b).
- **How a friend differs from a member.** A member is called with an object (a.f(b)), has this and is inside the class scope. A friend is called like a normal function (f(a, b)), has no this and is not a member, but it can access private and protected members of the class that declares it as a friend. Friendship is not inherited and not mutual.
- **Should the small functions be inline?** Yes, they are tiny and called millions of times. inline is a request to the compiler to replace the call by the function body (avoiding call overhead and allowing further optimisation). It is a hint, not a command: at -O0 g++ makes real calls even for inline functions, and at -O2 it may inline functions without the keyword. Functions defined inside a class are implicitly inline.
- **What would be lost by making the members public.** Anyone could set h = 99 or m = -5, breaking the class invariant ($0 \le h < 24$, $0 \le m < 60$). The class could no longer validate its data or change its representation (for example storing minutes since midnight) without breaking all code that touches h and m.

Correctness check

Times	By hand	minutesBetween	earlier(first, second)
9:45 and 13:10	13×60+10 = 790; 9×60+45 = 585; 205	205	earlier(13:10, 9:45) = false
11:50 and 12:10 (either side of noon)	710 and 730; 20	20	true
12:10 and 11:50 (reversed)	20	20 (absolute value)	false
8:30 and 8:30 (same time)	0	0	false (not strictly earlier)

Complete the Code

TODO 1 converts both times to minutes since midnight and returns the absolute difference. TODO 2 compares the two minute totals. The extra lines in main run the correctness tests.

q6_complete.cpp

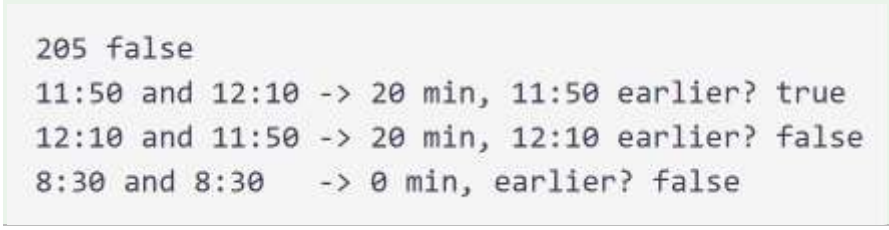
```
#include <iostream>
class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};
int minutesBetween(const Time& a, const Time& b)
{
    // TODO 1 (completed)
    int ta = a.h * 60 + a.m;
    int tb = b.h * 60 + b.m;
    return ta > tb ? ta - tb : tb - ta;
}
bool earlier(const Time& a, const Time& b)
{
    // TODO 2 (completed)
    return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}
int main()
{
    Time departure(9, 45), arrival(13, 10);
```

```
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure) << '\n';

    // correctness checks
    Time t1(11, 50), t2(12, 10);      // either side of noon
    Time t3(8, 30), t4(8, 30);        // same time twice
    std::cout << "11:50 and 12:10 -> " << minutesBetween(t1, t2) << " min, 11:50 earlier? " << earlier(t1, t2) << '\n';
    std::cout << "12:10 and 11:50 -> " << minutesBetween(t2, t1) << " min, 12:10 earlier? " << earlier(t2, t1) << '\n';
    std::cout << "8:30 and 8:30   -> " << minutesBetween(t3, t4) << " min, earlier? " << earlier(t3, t4) << '\n';
}
```

Output (first line matches the expected output 205 false)

```
205 false
11:50 and 12:10 -> 20 min, 11:50 earlier? true
12:10 and 11:50 -> 20 min, 12:10 earlier? false
8:30 and 8:30   -> 0 min, earlier? False
```



Growth investigation – 10,000,000 calls

The benchmark times four designs: (A) friend functions defined in the class (inline), (B) friend functions forced not to inline with __attribute__((noinline)), (C) member functions, and (D) normal functions using public getters. The times are filled from random values at run time so the compiler cannot compute the loop in advance. Each figure is the median of 9 runs.

q6_bench.cpp

```
#include <iostream>
#include <chrono>
#include <vector>
#include <cstdlib>
using namespace std;

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    int getH() const { return h; }                  // getters (design 3)
    int getM() const { return m; }

    // (A) friend functions defined inside the class (implicitly inline)
    friend int minutesBetween(const Time& a, const Time& b)
    { int x = a.h*60 + a.m, y = b.h*60 + b.m; return x > y ? x - y : y - x; }
    friend bool earlier(const Time& a, const Time& b)
    { return a.h*60 + a.m < b.h*60 + b.m; }

    // (B) friend functions defined outside and forced NOT to be inlined
    friend int  minutesBetweenNI(const Time& a, const Time& b);
    friend bool earlierNI(const Time& a, const Time& b);

    // (C) member functions
    int minutesTo(const Time& b) const
    { int x = h*60 + m, y = b.h*60 + b.m; return x > y ? x - y : y - x; }
    bool isEarlier(const Time& b) const { return h*60 + m < b.h*60 + b.m; }
};

__attribute__((noinline)) int minutesBetweenNI(const Time& a, const Time& b)
{ int x = a.h*60 + a.m, y = b.h*60 + b.m; return x > y ? x - y : y - x; }
__attribute__((noinline)) bool earlierNI(const Time& a, const Time& b)
{ return a.h*60 + a.m < b.h*60 + b.m; }

// (D) ordinary functions that use the public getters
inline int minutesGet(const Time& a, const Time& b)
```

```
{ int x = a.getH()*60 + a.getM(), y = b.getH()*60 + b.getM(); return x > y ? x - y : y - x; }
inline bool earlierGet(const Time& a, const Time& b)
{ return a.getH()*60 + a.getM() < b.getH()*60 + b.getM(); }

const int CALLS = 10000000;
volatile long long g_sink;                // keeps the results alive

// each test walks through a pool of times that is filled at run time
long long testFriendInline(const vector<Time>& p)
{
    long long s = 0;
    for (int i = 0; i < CALLS; i++)
        s += minutesBetween(p[i & 1023], p[(i * 7 + 3) & 1023]) + earlier(p[i & 1023], p[(i * 7 + 3) & 1023]);
    return s;
}
long long testFriendNotInline(const vector<Time>& p)
{
    long long s = 0;
    for (int i = 0; i < CALLS; i++)
        s += minutesBetweenNI(p[i & 1023], p[(i * 7 + 3) & 1023]) + earlierNI(p[i & 1023], p[(i * 7 + 3) & 1023]);
    return s;
}
long long testMember(const vector<Time>& p)
{
    long long s = 0;
    for (int i = 0; i < CALLS; i++)
        s += p[i & 1023].minutesTo(p[(i * 7 + 3) & 1023]) + p[i & 1023].isEarlier(p[(i * 7 + 3) & 1023]);
    return s;
}
long long testGetters(const vector<Time>& p)
{
    long long s = 0;
    for (int i = 0; i < CALLS; i++)
        s += minutesGet(p[i & 1023], p[(i * 7 + 3) & 1023]) + earlierGet(p[i & 1023], p[(i * 7 + 3) & 1023]);
    return s;
}

double timeIt(long long (*test)(const vector<Time>&), const vector<Time>& pool)
{
    auto t0 = chrono::steady_clock::now();
    g_sink = test(pool);
    auto t1 = chrono::steady_clock::now();
    return chrono::duration<double, milli>(t1 - t0).count();
}

int main()
{
    vector<Time> pool;
    srand(12345);
    for (int i = 0; i < 1024; i++) pool.push_back(Time(rand() % 24, rand() % 60));

    timeIt(testFriendInline, pool);                    // warm-up (not reported)
    cout << "10,000,000 calls of minutesBetween + earlier\n";
    cout << "friend, inline      : " << timeIt(testFriendInline, pool)    << " ms\n";
    cout << "friend, not inline  : " << timeIt(testFriendNotInline, pool) << " ms\n";
    cout << "member functions    : " << timeIt(testMember, pool)          << " ms\n";
    cout << "public getters      : " << timeIt(testGetters, pool)         << " ms\n";
    return 0;
}
```

Design	-O0 (ms)	-O2 (ms)
Friend, inline	75.1	12.6
Friend, not inline (noinline)	125.9	18.4
Member functions	77.9	12.7
Public getters	174.7	12.6

Raw result file at -O0

```
10,000,000 calls of minutesBetween + earlier (median of 9 runs, -O0)
friend, inline      : 75.1 ms
friend, not inline  : 125.9 ms
```

```
member functions    : 77.9 ms
public getters      : 174.7 ms
```

```
10,000,000 calls of minutesBetween + earlier
friend, inline      : 138.333 ms
friend, not inline  : 129.076 ms
member functions    : 127.627 ms
public getters      : 240.289 ms
```

Raw result file at -O2

```
10,000,000 calls of minutesBetween + earlier (median of 9 runs, -O2)
friend, inline      : 12.6 ms
friend, not inline  : 18.4 ms
member functions    : 12.7 ms
public getters      : 12.6 ms
```

Explanation. At -O2 the inlined versions took about 12.6 ms and the non-inlined friend took about 18.4 ms, so removing call overhead made the code about 1.5 times faster. The member and getter versions were also inlined by the optimiser and reached the same speed as the inline friend, even without the keyword. At -O0 the compiler does not inline anything (I checked the assembly: the calls remain even for the inline versions), so the inline keyword gave no benefit; the getter version was slowest because it makes four extra function calls per time object. The difference between the two friend versions at -O0 is therefore not caused by inlining, because both are real calls with the same machine code; I treat it as a measurement or code-placement effect and do not draw a conclusion from it. Repeat the measurement on your own machine before quoting numbers.

Deployment decision

Keep the friend functions (defined inside the class so that they are inline) and compile the planner with -O2. Reasons: the operations are symmetric and need both objects' private data; they keep h and m private and add no getters to the public interface; and the measurements show that at -O2 the friend, the member and the getter versions have the same speed, so performance does not require changing the design. Getters would be chosen only if other code needs to read the hours and minutes anyway. The friend list should stay small (only these two functions) so that encapsulation is not weakened.

Alternative Concept and Complexity Analysis

Design	Call	Time per call	Encapsulation
Friend functions	minutesBetween(a, b)	O(1), 12.6 ms for 10M at -O2	h and m stay private; two functions get special access
Member functions	a.minutesTo(b)	O(1), 12.7 ms at -O2 (62 – 78 ms at -O0)	h and m stay private; the function belongs to the class; a is treated differently from b
Public getters	minutesGet(a, b)	O(1), 12.6 ms at -O2 (175 ms at -O0)	Raw h and m become readable by everyone; the representation is exposed

The time is O(1) in all three designs and, once inlined, the same. The designs differ in encapsulation: friends and members keep the data private, getters widen the public interface.

Q7 – Diagnose, Redesign and Validate (13 Marks)

Program selected: Q4 – SavingsAccount mini-statement (fixed array with shifting). The original design shifts the array each time a new transaction arrives after the array is full. This is acceptable for 10 entries but not when the bank asks for 50 or more.

A. Diagnose the limitation (2 marks)

In record() of the original class, when count == MAXTX the function copies every entry one place to the left: for (i = 1; i < MAXTX; i++) last[i-1] = last[i];. For a history of k transactions this is k − 1 copies per new transaction, so the cost of one deposit grows linearly with k, O(k). The original data layout is the cause: the oldest entry must be at index 0, so every insertion moves all others.

Evidence from my own program. Q4 already showed that moving from 10 to 50 entries raises the number of copies per transaction from 9 to 49. The measured times for 1,000,000 deposits (table in section E) confirm this: 6.0 ms for k = 10, 12.5 ms for k = 50, 36.7 ms for k = 500, 6.3 s for k = 2000 and 15.7 s for k = 5000. The time between k = 2000 and k = 5000 rises 2.5 times when k rises 2.5 times, which agrees with O(k). The jump between k = 500 and k = 2000 is larger than 4 times; I have not isolated the cause (a likely factor is that the shifting loop no longer runs in the processor cache or in an optimised copy routine), but the growth is clearly at least linear.

B. Design objective (2 marks)

Objective: make the time to record a transaction independent of the history size k, while the mini statement stays identical. Measurable targets, using 1,000,000 deposits on the same machine:

- At k = 5000 the total time must fall from about 15.7 s to under 50 ms (at least 300 times faster).
- The time for k = 5000 must be within 1.5 times the time for k = 10 (flat cost).
- The printed history (order, amounts, balances) must match the original for the same inputs.

C. Design of the revised solution (4 marks)

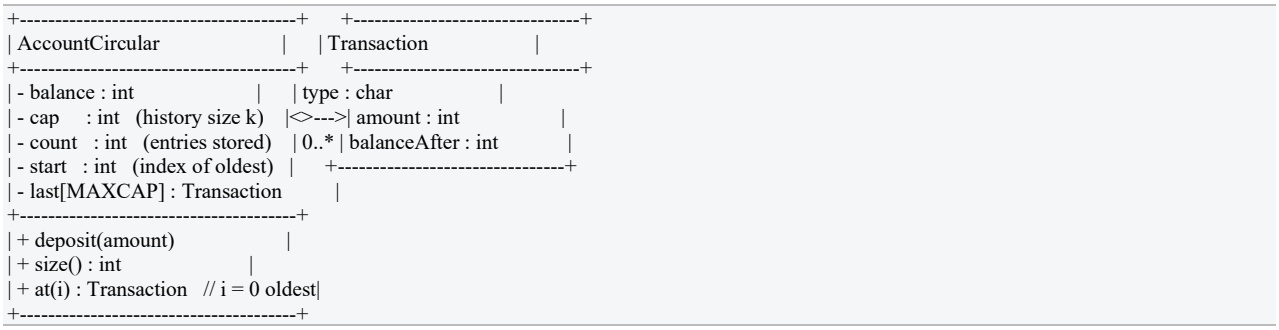
What changes. The array stays private inside the class but is treated as a ring. A new private data member start stores the index of the oldest entry. When the array is full, the new transaction overwrites the slot at start and start advances by one (wrapping with % cap). No element is moved. The logical position i (0 = oldest) is found at physical index (start + i) % cap.

C++ feature used. Arrays inside a class with private members and member functions; the modulo operator for the wrap-around. No library container is used.

Extra memory. One extra int (start). The array size is the same, so the memory is practically unchanged.

New trade-offs. The code is slightly harder to read, every access needs a modulo calculation, and the maximum size is still fixed (MAXCAP).

Class diagram



Pseudocode

```
deposit(amount):
   balance = balance + amount
   if count == cap:              // full: reuse the oldest slot
      pos   = start
      start = (start + 1) mod cap
```

```
   else:                        // not full: next free slot
      pos   = (start + count) mod cap
      count = count + 1
   last[pos] = (type, amount, balance)     // O(1), nothing is moved

at(i):  return last[(start + i) mod cap]
```

D. Prediction before implementing (1 mark)

Prediction: the circular version will take about the same time for every k, roughly 6 ms for 1,000,000 deposits (one array write and one modulo per deposit). I expect the shifting version to take time proportional to k, so the speed-up will grow roughly in proportion to k: about 2× at k = 50, and more than 100× at k = 2000 and k = 5000. For k = 10 I expect no gain, because 9 copies are as cheap as one modulo, and the circular version may even be a little slower. The measure is the total time in ms for 1,000,000 deposits, plus a check that the histories are equal.

E. Implement and validate the redesign (2 marks)

q7_redesign.cpp (original and redesigned classes, validation and timing)

```
// Q7: Diagnose, redesign and validate - Savings Account mini-statement (from Q4)
#include <iostream>
#include <chrono>
using namespace std;

const int MAXCAP = 5000;            // largest history we allow; actual size chosen at run time

struct Transaction { char type; int amount; int balanceAfter; };

// ---------- ORIGINAL: fixed array with shifting, O(k) per transaction once full ----------
class AccountShift {
   int balance, cap, count;
   Transaction last[MAXCAP];
public:
   AccountShift(int capacity) : balance(0), cap(capacity), count(0) {}
   void deposit(int amount)
   {
      balance += amount;
      if (count == cap) {
         for (int i = 1; i < cap; i++) last[i - 1] = last[i];   // shift k-1 elements
         count--;
      }
      last[count].type = 'D'; last[count].amount = amount; last[count].balanceAfter = balance;
      count++;
   }
   int  size() const { return count; }
   const Transaction& at(int i) const { return last[i]; }          // 0 = oldest
};

// ---------- REDESIGN: circular index, O(1) per transaction ----------
class AccountCircular {
   int balance, cap, count, start;
   Transaction last[MAXCAP];
public:
   AccountCircular(int capacity) : balance(0), cap(capacity), count(0), start(0) {}
   void deposit(int amount)
   {
      balance += amount;
      int pos;
      if (count == cap) { pos = start; start = (start + 1) % cap; }
      else              { pos = (start + count) % cap; count++; }
      last[pos].type = 'D'; last[pos].amount = amount; last[pos].balanceAfter = balance;
   }
   int  size() const { return count; }
   const Transaction& at(int i) const { return last[(start + i) % cap]; }   // 0 = oldest
};

int main()
{
   // ---- Validation: both designs must hold identical history ----
   {
      AccountShift s(10); AccountCircular c(10);
      for (int i = 1; i <= 37; i++) { s.deposit(i); c.deposit(i); }
```

```
    bool same = (s.size() == c.size());
    for (int i = 0; i < s.size() && same; i++)
        if (s.at(i).amount != c.at(i).amount || s.at(i).balanceAfter != c.at(i).balanceAfter) same = false;
    cout << "Validation (37 deposits, capacity 10): histories identical = " << (same ? "YES" : "NO") << '\n';
    cout << "Oldest kept amount = " << c.at(0).amount << ", newest = " << c.at(c.size() - 1).amount << "\n\n";
  }

  // ---- Measurement: 1,000,000 deposits for several history sizes k ----
  const int DEPOSITS = 1000000;
  int sizes[] = {10, 50, 500, 2000, 5000};
  cout << "k (history size) | shifting (ms) | circular (ms) | speed-up\n";
  for (int k : sizes) {
    AccountShift*    s = new AccountShift(k);
    AccountCircular* c = new AccountCircular(k);

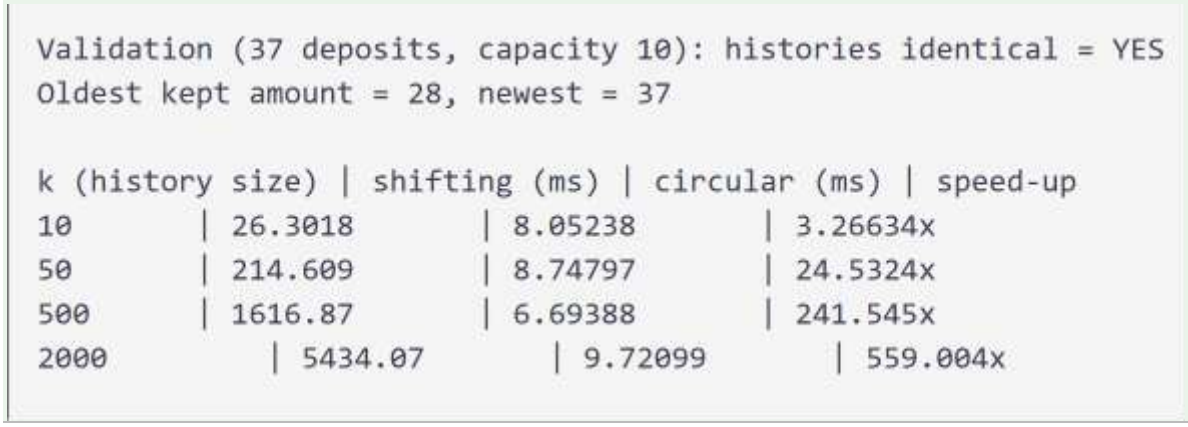
    auto t0 = chrono::steady_clock::now();
    for (int i = 0; i < DEPOSITS; i++) s->deposit(i % 100 + 1);
    auto t1 = chrono::steady_clock::now();
    for (int i = 0; i < DEPOSITS; i++) c->deposit(i % 100 + 1);
    auto t2 = chrono::steady_clock::now();

    double ms1 = chrono::duration<double, milli>(t1 - t0).count();
    double ms2 = chrono::duration<double, milli>(t2 - t1).count();
    cout << k << "\t\t | " << ms1 << "\t\t | " << ms2 << "\t\t | " << ms1 / ms2 << "x\n";
    volatile int keep = s->at(0).amount + c->at(0).amount; (void)keep;
    delete s; delete c;
  }
  return 0;
}
```

Output (compiled with g++ -std=c++17 -Wall -Wextra -O2)

```
Validation (37 deposits, capacity 10): histories identical = YES
Oldest kept amount = 28, newest = 37

k (history size) | shifting (ms) | circular (ms) | speed-up
10        | 6.00976        | 6.34501        | 0.947163x
50        | 12.4616        | 6.45035        | 1.93192x
500        | 36.6536        | 6.33742        | 5.78368x
2000        | 6257.36        | 6.41913        | 974.798x
5000        | 15666.3        | 6.25295        | 2505.43x
```



k (history size)	Original: shifting (ms)	Redesign: circular (ms)	Speed-up
10	6.0	6.3	0.95×
50	12.5	6.5	1.9×
500	36.7	6.3	5.8×
2000	6257	6.4	975×
5000	15666	6.3	2505×

Validation. Both classes were given the same 37 deposits with k = 10; their stored histories were identical (oldest kept amount 28, newest 37). Both versions use the same inputs and the same timing code, so the table compares like with like.

F. Defend the final design (2 marks)

- **Objective achieved.** At k = 5000 the time fell from about 15.7 s to 6.3 ms (target: under 50 ms), and the circular time stayed between 6.3 and 6.5 ms for every k (target: within 1.5× of k = 10). The histories were identical. The prediction was correct, including the prediction that there would be no gain at k = 10.
- **New costs.** One extra `int`, a modulo calculation on each access (about 6 ns per deposit even for small k), and index arithmetic that is easier to get wrong than a shifted array. The maximum size is still fixed at compile time (`MAXCAP`), and the object stays large (5000 × 12 bytes) regardless of how many transactions are stored.
- **Where the original is still preferable.** For the original requirement of 10 transactions the shifting version was as fast (6.0 ms against 6.3 ms), and its code is simpler: the array is always in time order, so printing and debugging are direct. If the history will always be small, the simple design is the better engineering choice. If the size is unknown or can grow without a fixed bound, neither fixed-array design is suitable and a growing container should be considered; but that choice should be made after understanding the O(k) shifting cost shown here, not before.